

AGENDA FOR THE DAY-5

LOGIC BUILDING

✖ Transfer statements

- Break
- Continue
- return

✖ == & equals()

✖ OOPS concepts

TRANSFER STATEMENTS

- ✗ Transfer statements are statements that change the normal flow of execution of a program.
- ✗ Instead of simply executing statements from top to bottom, they transfer control to another part of the program.
- ✗ break, continue, return

Statement	Purpose
break	Terminates a loop or switch
continue	Skips the current iteration of a loop
return	Exits from a method and optionally returns a value

FINAL

- ✖ final keyword is used to restrict or prevent changes. It can be applied to variables, methods, and classes.
- ✖ final Variable- Once a value is assigned to a final variable, it cannot be changed.

```
final int MAX_MARKS = 100;  
MAX_MARKS = 200; // ✖ Error
```

- ✖ A final variable must be initialized either:
- ✖ At the time of declaration, or
- ✖ Inside a constructor (for an instance final variable)

FINAL

- ✗ **final Method**– A final method cannot be overridden by a child class.

```
class Parent {  
    final void show() {  
        System.out.println("Parent show()");  
    }  
}
```

```
class Child extends Parent {  
    void show() { // ✗ Error  
        System.out.println("Child show()");  
    }  
}
```

Used when the parent class wants to ensure that a particular method's implementation cannot be changed by subclasses.

FINAL

- ✗ **final Class**- A final class cannot be inherited.

```
final class Vehicle {  
    void drive() {  
        System.out.println("Driving");  
    }  
}  
  
class Car extends Vehicle { // ✗ Error  
}
```

CONSTRUCTOR

- ✖ A constructor is a special member of a class that is used to initialize an object when the object is created.
- ✖ Constructor name must be same as class name
- ✖ A constructor should not have any return type like void, int, String, etc.
- ✖ Constructor is automatically called when an object is created.

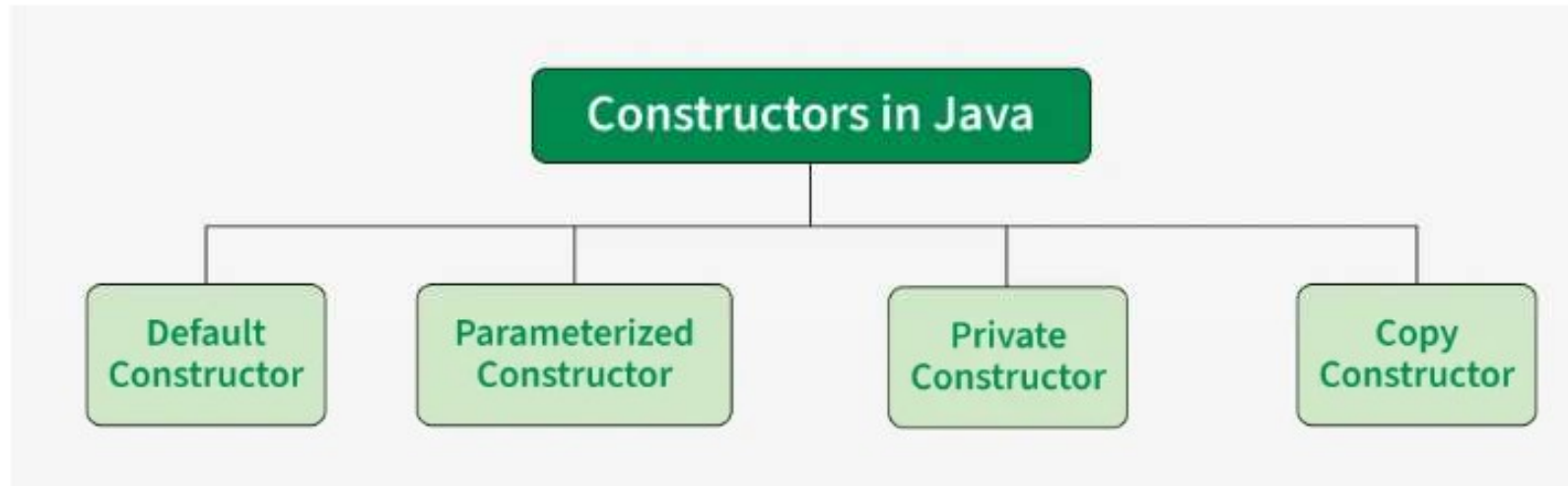
Syntax--

```
class Class-name{  
    Class-name() {  
        System.out.println("Constructor called");  
    }  
}
```

CONSTRUCTOR

Types of Constructors in Java

There are four types of constructors in Java



1. Default
2. Parameterized
3. Private
4. Copy

CONSTRUCTOR

1. Default Constructor

- ✖ A default constructor has no parameters. It's used to assign default values to an object. If no constructor is explicitly defined, Java provides a default constructor.
- ✖ It is not necessary to write a constructor for a class because the Java compiler automatically creates a default constructor (a constructor with no arguments) if your class doesn't have any.

```
Class Demo{  
    Demo( ){  
    }  
}
```


CONSTRUCTOR

2. Parameterized Constructor

- ✖ A constructor that has parameters is known as parameterized constructor.
- ✖ If we want to initialize fields of the class with our own values, then use a parameterized constructor.

```
Class Demo{  
    Demo( int a, String b){  
    }  
}
```

CONSTRUCTOR

3. Private Constructor

- ✗ A private constructor cannot be accessed from outside the class.

4. Copy Constructor-

- ✗ Unlike other constructors it is passed with another object which copies the data available from the passed object to the newly created object.

OOPS CONCEPTS IN JAVA

- ✗ OOPs stands for **Object-Oriented Programming System**
- ✗ Java is primarily an **object-oriented programming language**, where programs are designed using **classes and objects**.
- ✗ The four main pillars of OOPs are:
 1. **Encapsulation**
 2. **Inheritance**
 3. **Polymorphism**
 4. **Abstraction**

An object generally has two important things: State and behaviour

State means - Data/attributes of an object. For example- String name; int age;

Behavior- Actions an object can perform.

```
void study() {  
    System.out.println("Studying");  
}
```

OOPS CONCEPTS IN JAVA

Encapsulation means wrapping data and methods together into a single unit and controlling access to the data.

- ✗ Usually achieved using: private variables & public getter/setter methods

Inheritance allows one class to acquire properties and behaviors of another class.

Polymorphism means "many forms".

The same method/interface can behave differently depending on the situation.

Java mainly has:

- ✗ Compile-time polymorphism
- ✗ Runtime polymorphism

OOPS CONCEPTS IN JAVA

Abstraction– Abstraction means hiding implementation details and showing only essential functionality.

Java provides abstraction mainly through:

- ✗ Abstract classes
- ✗ Interfaces

ARRAY

Collection of same data types stored in the contiguous memory location.
It make use of indexing to access or store elements.

```
int arr1[] = new int[5];
```

```
int arr2[] = {10,15,28,23,24}
```

== && EQUALS()

Difference between == and equals()

== → compares references

equals() → compares content